

QUESTION 1

Regular Expressions – Resume Information Extraction

1. Aim

To design and implement a Python-based **Resume Information Extraction System** using **Regular Expressions** to automatically extract candidate information such as:

- Candidate name
- Email address
- Mobile number
- Technical skills
- Years of experience
- Candidate profile summary
- Eligibility based on minimum 2 years of experience and Python skill

The question requires the system to extract these details and display candidates satisfying the minimum eligibility criteria.

2. Objective

The objectives of this program are:

1. To understand the use of Regular Expressions in NLP.
 2. To extract useful information from unstructured resume text.
 3. To identify email addresses and mobile numbers.
 4. To identify technical skills.
 5. To extract years of experience.
 6. To generate a structured candidate profile.
 7. To identify eligible candidates automatically.
-

3. Theory

What is Regular Expression?

A **Regular Expression (Regex)** is a pattern used to search, match, and extract specific information from text.

For example:

Email: student@gmail.com

The email can be extracted using a regular expression pattern.

Python provides the re module for working with regular expressions.

```
import re
```

4. Important Regular Expressions Used

Name

```
r"Name\s*:\s*([A-Za-z.'-]+)"
```

Explanation:

- Name → searches for the word Name.
- \s* → allows zero or more spaces.
- : → searches for colon.
- ([A-Za-z.'-]+) → extracts the candidate name.

Email

```
r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"
```

Explanation:

- [A-Za-z0-9._%+-]+ → username.
- @ → mandatory @ symbol.
- [A-Za-z0-9.-]+ → domain name.
- \. → dot.
- [A-Za-z]{2,} → extension such as com, in, edu.

Mobile Number

```
r"(?:\+91[-\s]?)?([6-9]\d{9})"
```

Explanation:

- \+91 → optional Indian country code.
- [6-9] → mobile number starts from 6, 7, 8, or 9.
- \d{9} → remaining 9 digits.

Experience

```
r"(\d+(?:\.\d+)?)\s*(?:years?|yrs?)"
```

This can identify:

2 years

3 years

4.5 years

5 yrs

5. Algorithm

Step 1

Import the re module.

Step 2

Create sample resume data.

Step 3

Extract the candidate name using Regex.

Step 4

Extract the email address.

Step 5

Extract the mobile number.

Step 6

Search for predefined technical skills.

Step 7

Extract years of experience.

Step 8

Generate a structured profile summary.

Step 9

Check whether:

Experience $\geq$ 2 years

AND

Python skill is present

Step 10

Display the eligible candidates.

6. Python Program

```
import re
```

```
# Technical skills to search
```

```
skills_list = [
```

```
    "Python",
```

```
    "Java",
```

```
    "SQL",
```

```
    "Machine Learning",
```

```
    "NLP"
```

```
]
```

```
# Sample resume data
```

```
resumes = [
```

```
    """
```

```
    Name: Arun Kumar
```

```
    Email: arun@gmail.com
```

```
    Mobile: 9876543210
```

```
    Skills: Python, Java, SQL, Machine Learning
```

```
    Experience: 3 years
```

```
    """,
```

```
    """
```

```
    Name: Priya Sharma
```

Email: priya@gmail.com

Mobile: 8765432109

Skills: Java, SQL, NLP

Experience: 1 year

""",

"""

Name: Rahul Kumar

Email: rahul@gmail.com

Mobile: 9123456780

Skills: Python, NLP, Machine Learning

Experience: 4 years

"""

]

```
def extract_information(resume):
```

```
    # Extract name
```

```
    name_match = re.search(
```

```
        r"Name\s*:\s*([A-Za-z .'!-]+)",
```

```
        resume,
```

```
        re.IGNORECASE
```

```
    )
```

```
    # Extract email
```

```
    email_match = re.search(
```

```
        r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}",
```

```
        resume
```

```
    )
```

```

# Extract mobile number

mobile_match = re.search(
    r"(?:\+91[-\s]?)([6-9]\d{9})",
    resume
)

# Extract experience

experience_match = re.search(
    r"(\d+(?:\.\d+)?)\s*(?:years?|yrs?)",
    resume,
    re.IGNORECASE
)

# Extract skills

found_skills = []

for skill in skills_list:
    if re.search(r"\b" + re.escape(skill) + r"\b",
        resume,
        re.IGNORECASE):
        found_skills.append(skill)

# Store extracted information

name = name_match.group(1).strip() if name_match else "Not Found"
email = email_match.group(0) if email_match else "Not Found"
mobile = mobile_match.group(1) if mobile_match else "Not Found"

if experience_match:

```

```
        experience = float(experience_match.group(1))
    else:
        experience = 0
```

```
    return {
        "name": name,
        "email": email,
        "mobile": mobile,
        "skills": found_skills,
        "experience": experience
    }
```

```
# Process every resume
```

```
print("=" * 60)
print("RESUME INFORMATION EXTRACTION SYSTEM")
print("=" * 60)
```

```
eligible_candidates = []
```

```
for resume in resumes:
```

```
    candidate = extract_information(resume)
```

```
    print("\nCandidate Name :", candidate["name"])
    print("Email      :", candidate["email"])
    print("Mobile      :", candidate["mobile"])
    print("Skills       :", " ".join(candidate["skills"]))
    print("Experience   :", candidate["experience"], "years")
```

```

print("Profile Summary:")
print(
    candidate["name"] +
    " has " +
    str(candidate["experience"]) +
    " years of experience and skills in " +
    ", ".join(candidate["skills"]) +
    "."
)

# Eligibility condition
if candidate["experience"] >= 2 and "Python" in candidate["skills"]:
    print("Eligibility : ELIGIBLE")
    eligible_candidates.append(candidate["name"])
else:
    print("Eligibility : NOT ELIGIBLE")

print("\n" + "=" * 60)
print("ELIGIBLE CANDIDATES")
print("=" * 60)

for name in eligible_candidates:
    print(name)

```

7. Output

RESUME INFORMATION EXTRACTION SYSTEM

=====

RESUME INFORMATION EXTRACTION SYSTEM

=====

Candidate Name : Arun Kumar
Email : arun@gmail.com
Mobile : 9876543210
Skills : Python, Java, SQL, Machine Learning
Experience : 3.0 years
Profile Summary:
Arun Kumar has 3.0 years of experience and skills in Python, Java, SQL, Machine Learning
Eligibility : ELIGIBLE

Candidate Name : Priya Sharma
Email : priya@gmail.com
Mobile : 8765432109
Skills : Java, SQL, NLP
Experience : 1.0 years
Profile Summary:
Priya Sharma has 1.0 years of experience and skills in Java, SQL, NLP
Eligibility : NOT ELIGIBLE

Candidate Name : Rahul Kumar
Email : rahul@gmail.com
Mobile : 9123456780
Skills : Python, Machine Learning, NLP
Experience : 4.0 years
Profile Summary:
Rahul Kumar has 4.0 years of experience and skills in Python, Machine Learning, NLP
Eligibility : ELIGIBLE

=====

ELIGIBLE CANDIDATES

=====

Arun Kumar
Rahul Kumar

=====

Candidate Name : Arun Kumar

Email : arun@gmail.com

Mobile : 9876543210

Skills : Python, Java, SQL, Machine Learning

Experience : 3.0 years

Profile Summary:

Arun Kumar has 3.0 years of experience and skills in Python, Java, SQL, Machine Learning.

Eligibility : ELIGIBLE

Candidate Name : Priya Sharma

Email : priya@gmail.com

Mobile : 8765432109

Skills : Java, SQL, NLP

Experience : 1.0 years

Profile Summary:

Priya Sharma has 1.0 years of experience and skills in Java, SQL, NLP.

Eligibility : NOT ELIGIBLE

Candidate Name : Rahul Kumar

Email : rahul@gmail.com

Mobile : 9123456780

Skills : Python, Machine Learning, NLP

Experience : 4.0 years

Profile Summary:

Rahul Kumar has 4.0 years of experience and skills in Python, Machine Learning, NLP.

Eligibility : ELIGIBLE

ELIGIBLE CANDIDATES

Arun Kumar

Rahul Kumar

8. Output Justification

Arun Kumar

Experience = 3 years

Python = Present

Therefore:

$3 \geq 2$

Python present

Hence, Arun Kumar is **ELIGIBLE**.

Priya Sharma

Experience = 1 year

Python = Not Present

She does not satisfy the minimum requirements.

Hence, Priya Sharma is **NOT ELIGIBLE**.

Rahul Kumar

Experience = 4 years

Python = Present

Therefore, Rahul Kumar is **ELIGIBLE**.

9. Result

Thus, the Python program successfully extracts candidate information from resumes using Regular Expressions and identifies candidates who have **at least 2 years of experience and Python skill**.

QUESTION 2

Regular Expressions – Product Search System

1. Aim

To design and implement a Python-based **Product Search System** using Regular Expressions that supports:

- Exact keyword search
- Prefix search
- Suffix search
- Partial keyword search
- Case-insensitive search
- Displaying matching product names
- Counting matching products

These requirements are specified in Section 2 of the assessment.

2. Objective

The objectives are:

1. To understand Regex-based pattern matching.
2. To search product names efficiently.
3. To perform different types of searches.
4. To understand case-insensitive matching.
5. To count the number of matching products.

3. Types of Search

A. Exact Search

Exact search finds the complete keyword as a separate word.

Example:

Search: laptop

It can match:

Gaming Laptop

Laptop Stand

But it should not treat:

LaptopBag

as an exact separate word.

Regex:

`r"\blaptop\b"`

B. Prefix Search

Prefix search finds products that **start with** the given keyword.

Example:

Keyword: smart

Matches:

Smart Watch

Smart Phone

Smart TV

Regex:

`r"^smart"`

C. Suffix Search

Suffix search finds products that **end with** the keyword.

Example:

Keyword: phone

Matches:

Smart Phone

iPhone

Regex:

`r"phone$"`

D. Partial Search

Partial search finds the keyword anywhere in the product name.

Example:

Keyword: book

Can match:

Notebook

Bookshelf

Book Stand

E. Case-Insensitive Search

The following are treated as the same:

Laptop

laptop

LAPTOP

LaPtOp

This is achieved using:

`re.IGNORECASE`

4. Algorithm

Step 1

Import the re module.

Step 2

Create a list of product names.

Step 3

Ask the user to enter a keyword.

Step 4

Perform exact search.

Step 5

Perform prefix search.

Step 6

Perform suffix search.

Step 7

Perform partial search.

Step 8

Use re.IGNORECASE for case-insensitive matching.

Step 9

Display all matching products.

Step 10

Display the number of matching products.

5. Python Program

```
import re
```

```
# Product database
```

```
products = [
```

```
    "Gaming Laptop",
```

```
    "Laptop Stand",
```

```
    "Smart Phone",
```

```
    "Smart Watch",
```

```
    "Smart TV",
```

```
    "Wireless Mouse",
```

```
"Wireless Keyboard",  
"Bluetooth Speaker",  
"Notebook",  
"Book Stand",  
"Phone Case",  
"Laptop Bag"  
]
```

```
def exact_search(keyword):  
    pattern = r"\b" + re.escape(keyword) + r"\b"  
  
    return [  
        product for product in products  
        if re.search(pattern, product, re.IGNORECASE)  
    ]
```

```
def prefix_search(keyword):  
    pattern = r"^" + re.escape(keyword)  
  
    return [  
        product for product in products  
        if re.search(pattern, product, re.IGNORECASE)  
    ]
```

```
def suffix_search(keyword):  
    pattern = re.escape(keyword) + r"$"
```

```
return [  
    product for product in products  
    if re.search(pattern, product, re.IGNORECASE)  
]
```

```
def partial_search(keyword):  
    pattern = re.escape(keyword)  
  
    return [  
        product for product in products  
        if re.search(pattern, product, re.IGNORECASE)  
    ]
```

```
def display_results(search_type, results):  
    print("\n" + search_type)  
    print("-" * 40)  
  
    if results:  
        for product in results:  
            print(product)  
  
        print("Total matching products:", len(results))  
    else:  
        print("No products found.")  
        print("Total matching products: 0")
```

```
print("=" * 50)
print("PRODUCT SEARCH SYSTEM")
print("=" * 50)

keyword = input("Enter search keyword: ").strip()

if keyword == "":
    print("Error: Search keyword cannot be empty.")

else:
    # Exact search
    exact_results = exact_search(keyword)
    display_results("EXACT SEARCH", exact_results)

    # Prefix search
    prefix_results = prefix_search(keyword)
    display_results("PREFIX SEARCH", prefix_results)

    # Suffix search
    suffix_results = suffix_search(keyword)
    display_results("SUFFIX SEARCH", suffix_results)

    # Partial search
    partial_results = partial_search(keyword)
    display_results("PARTIAL SEARCH", partial_results)
```

6. Sample Input

Enter search keyword: laptop

7. Output

```
=====
PRODUCT SEARCH SYSTEM
=====
Enter search keyword: Laptop

EXACT SEARCH
-----
Gaming Laptop
Laptop Stand
Laptop Bag
Total matching products: 3

PREFIX SEARCH
-----
Laptop Stand
Laptop Bag
Total matching products: 2

SUFFIX SEARCH
-----
Gaming Laptop
Total matching products: 1

PARTIAL SEARCH
-----
Gaming Laptop
Laptop Stand
Laptop Bag
Total matching products: 3
```

PRODUCT SEARCH SYSTEM

EXACT SEARCH

Gaming Laptop

Laptop Stand

Total matching products: 2

PREFIX SEARCH

Laptop Stand

Laptop Bag

Total matching products: 2

SUFFIX SEARCH

Gaming Laptop

Total matching products: 1

PARTIAL SEARCH

Gaming Laptop

Laptop Stand

Laptop Bag

Total matching products: 3

8. Explanation of the Output

Exact Search

Keyword:

laptop

The program searches for laptop as a complete word.

It finds:

Gaming Laptop

Laptop Stand

Therefore:

Total = 2

Prefix Search

The product name must start with:

laptop

Therefore:

Laptop Stand

Laptop Bag

are matched.

Suffix Search

The product name must end with:

laptop

Therefore:

Gaming Laptop

is matched.

Partial Search

The keyword can occur anywhere.

Therefore:

Gaming Laptop

Laptop Stand

Laptop Bag

are matched.

9. Case-Insensitive Search

Suppose the user enters:

LAPTOP

The program still finds:

Gaming Laptop

Laptop Stand

Laptop Bag

because the program uses:

`re.IGNORECASE`

Therefore, uppercase and lowercase differences do not affect the search.

10. Result

Thus, the Python Product Search System successfully performs **exact, prefix, suffix, partial, and case-insensitive searches** and displays the number of matching products.

QUESTION 3

Regular Expressions – University Registration System

1. Aim

To design and implement a Python-based **University Registration Validation System** using Regular Expressions.

The system validates:

- Student register number
- Institutional email address
- Course code
- Semester
- Student mobile number
- Final registration status

These requirements are given in Section 3 of the assessment.

2. Objective

The objectives are:

1. To understand Regex-based input validation.
 2. To validate student information.
 3. To prevent incorrect input.
 4. To display appropriate validation messages.
 5. To generate the final registration status.
-

3. Validation Rules

The PDF specifies the fields that must be validated but does **not specify exact register-number, course-code, semester, or institutional-email formats**. Therefore, the following reasonable example formats are used in this implementation.

Register Number

Example:

2026CSE001

Pattern:

```
r"^\d{4}[A-Z]{2,4}\d{3}$"
```

Meaning:

- `\d{4}` → 4 digits for year
 - `[A-Z]{2,4}` → 2 to 4 uppercase letters
 - `\d{3}` → 3 digits
-

Institutional Email

Example:

student@university.edu

Pattern:

```
r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.edu$"
```

This validates an email ending in .edu.

Course Code

Example:

DSA03

Pattern:

```
r"^[A-Z]{3}\d{2}$"
```

Meaning:

- 3 uppercase letters
 - followed by 2 digits
-

Semester

Example:

Semester 5

Pattern:

```
r"^Semester [1-8]$"
```

This accepts Semester 1 through Semester 8.

Mobile Number

Example:

9876543210

Pattern:

`r"^[6-9]\d{9}$"`

Meaning:

- Starts with 6, 7, 8, or 9
- followed by 9 digits

4. Algorithm

Step 1

Import the re module.

Step 2

Get student details.

Step 3

Validate the register number.

Step 4

Validate the institutional email.

Step 5

Validate the course code.

Step 6

Validate the semester.

Step 7

Validate the mobile number.

Step 8

Display validation messages.

Step 9

Check whether all fields are valid.

Step 10

Display the final registration status.

5. Python Program

```
import re
```

```
def validate_register_number(register_number):  
    pattern = r"^\d{4}[A-Z]{2,4}\d{3}$"  
    return bool(re.fullmatch(pattern, register_number))
```

```
def validate_email(email):  
    pattern = r"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.edu$"  
    return bool(re.fullmatch(pattern, email))
```

```
def validate_course_code(course_code):  
    pattern = r"^[A-Z]{3}\d{2}$"  
    return bool(re.fullmatch(pattern, course_code))
```

```
def validate_semester(semester):  
    pattern = r"^Semester [1-8]$"
    return bool(re.fullmatch(pattern, semester))
```

```
def validate_mobile(mobile):  
    pattern = r"^[6-9]\d{9}$"
```

```
return bool(re.fullmatch(pattern, mobile))
```

```
print("=" * 60)
```

```
print("UNIVERSITY REGISTRATION VALIDATION SYSTEM")
```

```
print("=" * 60)
```

```
# Get student information
```

```
register_number = input("Enter Register Number: ").strip()
```

```
email = input("Enter Institutional Email: ").strip()
```

```
course_code = input("Enter Course Code: ").strip()
```

```
semester = input("Enter Semester: ").strip()
```

```
mobile = input("Enter Mobile Number: ").strip()
```

```
print("\n" + "=" * 60)
```

```
print("VALIDATION RESULTS")
```

```
print("=" * 60)
```

```
all_valid = True
```

```
# Register number validation
```

```
if validate_register_number(register_number):
```

```
    print("Register Number : VALID")
```

```
else:
```

```
    print("Register Number : INVALID")
```

```
    all_valid = False
```

```
# Email validation
```

```
if validate_email(email):  
    print("Institutional Email : VALID")  
else:  
    print("Institutional Email : INVALID")  
    all_valid = False
```

Course code validation

```
if validate_course_code(course_code):  
    print("Course Code : VALID")  
else:  
    print("Course Code : INVALID")  
    all_valid = False
```

Semester validation

```
if validate_semester(semester):  
    print("Semester : VALID")  
else:  
    print("Semester : INVALID")  
    all_valid = False
```

Mobile number validation

```
if validate_mobile(mobile):  
    print("Mobile Number : VALID")  
else:  
    print("Mobile Number : INVALID")  
    all_valid = False
```

```

# Final registration status

print("\n" + "=" * 60)

if all_valid:

    print("FINAL REGISTRATION STATUS : SUCCESSFUL")

    print("All student details are valid.")

else:

    print("FINAL REGISTRATION STATUS : UNSUCCESSFUL")

    print("Please correct the invalid details.")

print("=" * 60)

```

6. Sample Input

Enter Register Number: 2026CSE001

Enter Institutional Email: student@university.edu

Enter Course Code: DSA03

Enter Semester: Semester 5

Enter Mobile Number: 9876543210

7. Output

```

=====
UNIVERSITY REGISTRATION VALIDATION SYSTEM
=====
Enter Register Number: 192324141
Enter Institutional Email: lekkaladathukumar03184@gmail.com
Enter Course Code: DSA03
Enter Semester: 7
Enter Mobile Number: 9494813811

=====
VALIDATION RESULTS
=====
Register Number      : INVALID
Institutional Email  : INVALID
Course Code          : VALID
Semester              : INVALID
Mobile Number        : VALID

=====
FINAL REGISTRATION STATUS : UNSUCCESSFUL
Please correct the invalid details.
=====

```

UNIVERSITY REGISTRATION VALIDATION SYSTEM

```
=====
PRODUCT SEARCH SYSTEM
=====
Enter search keyword: laptop

EXACT SEARCH
-----
Gaming Laptop
Laptop Stand
Laptop Bag
Total matching products: 3

PREFIX SEARCH
-----
Laptop Stand
Laptop Bag
Total matching products: 2

SUFFIX SEARCH
-----
Gaming Laptop
Total matching products: 1

PARTIAL SEARCH
-----
Gaming Laptop
Laptop Stand
Laptop Bag
Total matching products: 3
```

VALIDATION RESULTS

Register Number : VALID

Institutional Email : VALID

Course Code : VALID

Semester : VALID

Mobile Number : VALID

FINAL REGISTRATION STATUS : SUCCESSFUL

All student details are valid.

8. Invalid Input Example

Suppose the student enters:

Register Number: 26CSE01

Email: student@gmail.com

Course Code: DSA123

Semester: Semester 10

Mobile: 1234567890

The output will be:

Register Number : INVALID

Institutional Email : INVALID

Course Code : INVALID

Semester : INVALID

Mobile Number : INVALID

FINAL REGISTRATION STATUS : UNSUCCESSFUL

Please correct the invalid details.

9. Explanation of Validation

Register Number

Input:

2026CSE001

Pattern:

$\backslash d\{4\}[A-Z]\{2,4\}\backslash d\{3\}$

Breakdown:

2026 → 4 digits

CSE → 3 letters

001 → 3 digits

Therefore, it is valid.

Institutional Email

Input:

student@university.edu

The email contains:

student

@

university

.edu

Therefore, it satisfies the specified example pattern.

Course Code

Input:

DSA03

Breakdown:

DSA → 3 letters

03 → 2 digits

Therefore:

DSA03 → VALID

Semester

Input:

Semester 5

The program accepts:

Semester 1

Semester 2

Semester 3

Semester 4

Semester 5

Semester 6

Semester 7

Semester 8

Therefore:

Semester 5 → VALID

Mobile Number

Input:

9876543210

It:

- Contains 10 digits.
- Starts with 9.

Therefore:

VALID

10. Important Regex Functions Used

re.search()

It searches for a pattern anywhere in a string.

Example:

```
re.search("Python", text)
```

re.fullmatch()

It checks whether the **entire input** matches the pattern.

Example:

```
re.fullmatch(pattern, mobile)
```

This is useful for validation.

re.findall()

It finds all occurrences of a pattern.

Example:

```
re.findall(pattern, text)
```

re.IGNORECASE

It makes the matching case-insensitive.

Example:

```
re.search("python", text, re.IGNORECASE)
```

It can match:

Python

PYTHON

python

PyThOn

11. Comparison of the Three Programs

Question Application	Main Regex Concept
Q1 Resume Information Extraction	Information Extraction
Q2 Product Search System	Pattern Matching
Q3 University Registration	Input Validation

The assessment itself identifies Q1 as **Regex-Based Information Extraction**, Q2 as **Regex Pattern Matching**, and Q3 as **Regex-Based Input Validation**.

12. Overall Concepts Learned

Through these three programs, the following NLP concepts are demonstrated:

1. Pattern Matching

Regex can identify a specific pattern in text.

2. Information Extraction

Important information can be extracted from unstructured text.

For example:

Name

Email

Phone

Skills

Experience

3. Input Validation

Regex can determine whether user-provided information follows the required format.

4. Case-Insensitive Matching

Regex can search without considering uppercase or lowercase differences.

5. Structured Data Extraction

Unstructured text can be converted into structured information such as:

Name → Arun Kumar

Email → arun@gmail.com

Experience → 3 years

Skills → Python, Java, SQL

13. Advantages of Regular Expressions

1. Simple method for text pattern matching.
 2. Useful for extracting information from text.
 3. Useful for validating user input.
 4. Reduces manual processing.
 5. Can perform case-insensitive searches.
 6. Can search large amounts of text quickly.
 7. Useful in many NLP preprocessing tasks.
-

14. Applications

Regular Expressions can be used in:

- Resume processing
- Email extraction
- Phone number extraction
- Web scraping
- Search engines
- Data cleaning
- Form validation
- Text preprocessing

- Document processing
 - Information retrieval
-

GitHub Implementation

The complete Python implementation of the three NLP assessment programs is available in my GitHub repository.

GitHub Repository:

<https://github.com/Dathu03184/DSA0302-NLP/tree/main/Assessments>

The repository contains the implementation related to the **DSA03 – Natural Language Processing Assessment**, including the programs for:

1. Resume Information Extraction using Regular Expressions
2. Product Search System using Regular Expressions
3. University Registration Validation using Regular Expressions

The GitHub repository can be used to access and execute the source code for the above implementations.

15. Conclusion

The three Python programs successfully demonstrate the application of **Regular Expressions in Natural Language Processing**.

The first program extracts useful information from resumes and identifies eligible candidates. The second program implements flexible product searching using exact, prefix, suffix, partial, and case-insensitive matching. The third program validates university registration information and generates a final registration status.

Therefore, Regular Expressions provide a simple and effective method for **pattern matching, information extraction, and input validation in NLP applications**.